Ariel Purnama Muharram
23521013

Python program => Sequence of definitions and commands
- definitions → evaluated
- commands → executed

Objects
- → Scalar
- → Non-scalar => Have internal structure

<object> <operator> <object>
(expression)

typing
- → Strong typing
- → Weak typing
- → Static typing
- → Dynamic typing

Static typing => Variable types are known at compile time.

Dynamic typing => # Compiler or an interpreter assigns a type to all the variables at run time
The type of a variable is decided based on its value

Input: >> input("Type something here...")

Control flow: # >> if: ... else: ...
>> if: ... elif: ... else: ...
>> while <condition>: <expression>
>> for <variable> in range(<num>): <expression>

Exit loop → break

Functions → mechanism to abstraction and decomposition
- → documentation in docstrings
- → Characteristic:
  - Has name
  - Has parameters
  - Has a docstring (optional)
  - Has a body
  - Return something

Assert

>> assert [kondisi benar], [pesan error]

>> assert int("lore ipsum"), "bukan angka"

Error Catch

```
try:
    [Block of codes]
except:
    [Block of codes]
else:
    [Block of codes]
finally:
    [Block of codes]
```

Error in Python
SyntaxError
NameError => Not found
AttributeError => wrong reference
TypeError => wrong type
ValueError => illegal value
IOError

Signal error
>> raise Exception("lore ipsum")

Keywords
- try
- except → catch error
- else → executed when no error
- finally → executed whether there is error or not

OOP

__ → make private attribute
- → access
  - _Cls__name
  - Self._Cls__name

~~def~~ class Car:
    def __init__(self):
        ~~super(Car).__init__()~~
        super(Car, self).__init__()

class Toyota(Car):
    def __init__(self):
        Car.__init__(self)

override class
- __str__(self) → "print" statement
- __add__(self) → "add" operator
- __eq__(self) → "equal" operator

Import

>> import <module-name>
>> from <module-name> import (*) → all class
>> impor <module-name> as (<something>) → aliasing

File

>> with open(<name>, ("w") as <name_handle>:
- w: write
- r: read
- a: append

<expression> → name_handle.~~read~~ read()
- write()
- readline()
- writelines()
- readlines()

Compound data types
- → list
- → tuple
- → dictionary

String → sequence of char
- → Immutable
- → ~~split~~ split(<delimiter>) → memisahkan string ke dalam list

Tuple: Ordered sequence of elements, (immutable)
- → swap: (x, y) = (y, x)

List: Ordered sequence of elements, (mutable)
- → append() => menambahkan item ke dalam list
- → (+) => List concatenation
- → remove(<item>) => menghapus item
- → del L[<idx>] => menghapus item berdasarkan index
- → pop() => ~~menga~~ mengeluarkan item terakhir
- → " ".join(L) => menggabungkan list

Scanned with CamScanner

~~Sort~~ Sort vs Sorted

- Sorted → returns sorted list array
  - >> sorted (L)
- Sort → Mutate list
  - >> L. sort()
  - >> L. reverse()

Recursive process — consists of
- → Base case
- → Recursive step

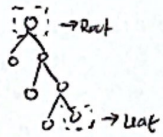
Algorithm complexity
- → mengukur seberapa efektif dan efisien sebuah algoritma
- → solusi: Order of Growth
  - → Big O notation

Dynamic Programming → Memoisasi
- Melakukan memoisasi terhadap proses yang berulang

Graph
- → Directed
- → Undirected
- → Weighted
- → Unweighted

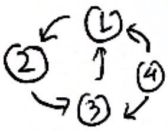
cyclic graph

Tree → merupakan graf khusus
- → bersifat directed
- → Single path



Building graphs
- → Nodes
- → Edges
- → Stitching process

Digraph → Directed Graph



	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	0	0	0
4	1	0	1	0

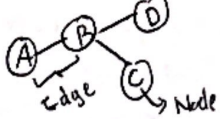
Source ↓

destination →

Adjacency matrix ⇝ memory intense

BFS tidak bisa diadopsi untuk weighted karena tidak mencoba seluruh kemungkinan.

Algoritma Greedy ⇝ Perlu dilakukan sort !!
- → Setiap menemukan goal maka dia langsung berhenti, terlepas dari semua solusi sudah dicoba atau belum.



~~Class~~

```
class Node (object):
    def __init__(self, name):
        self.name = name
    def getName(self): return self.name
    def __str__(self): return self.name

class Edge (object):
    def __init__(self, src, dest):
        self.src = src
        self.dest = dest
    def getSource(self): return self.src
    def getDestination(self): return self.dest
    def __str__(self): return self.src.getName() + '->'
                       + self.dest.getName()

class Digraph (object):
    def __init__(self):
        self.edges = { }
    def addNode(self, node):
        if node in self.edges:
            raise ValueError('Duplicated node')
        else:
            self.edges[node] = [ ]
    def addEdge(self, edge):
        src = edge.getSource()
        dest = edge.getDestination()
        if not (src in self.edges and dest in self.edges):
            raise ValueError('Node not in graph')
        self.edges[src].append(dest)

class Graph (Digraph):
    def addEdge(self, edge):
        Digraph.addEdge(self, edge)
        rev = Edge(edge.getDestination(), edge.getSource())
        Digraph.addEdge(self, rev)
```

→ Liskov Substitution Principle in SOLID

Measuring algorithm complexity:

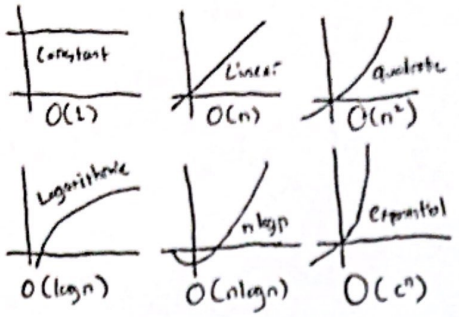
Using timer:
- Varies between implementations X
- Varies between computers X
- Time is not predictable on small inputs X
- Can not express between inputs and time X

Using counting operations:
- Depends on implementations X
- No clear definition of which operations to count X
- Can not express between inputs and time X

Order of Growth ✓
- → using Big-O notation
  - → Measures an upper bound on the asymptotic growth
  - → describe worst case

Scanned with CamScanner

Types of Orders of Growth



Greedy algorithm
- ↳ by Value
- ↳ by weight
- ↳ by ~~cost~~ density

```
def greedy (items, maxCost, keyFunction):
    itemsCopy = sorted(items, key=keyFunction, reverse=True)
    result = [ ]
    totalValue, totalCost = 0.0, 0.0
    for i in range(len(itemsCopy)):
        if (totalCost + itemsCopy[i].getCost()) <= maxCost:
            result.append (itemsCopy[i])
            totalCost += itemsCopy[i].getCost()
            totalValue += itemsCopy[i].getValue()
    return (result, totalValue)
```

```
# Memo (dynamic programing) used only by recursive calls
def fastFib(n, memo={}):
    if n==0 or n==1:
        return 1
    try:
        return memo[n]
    except KeyError:
        result = fastFib(n-1, memo) + fastFib(n-2, memo)
        memo[n] = result
        return result
```

~~slid~~ • ~~S[start~~ S[idx-start, idx-end, step]

S[3:6]

Aspects of language:
- ↳ Syntax : Structure of word
- ↳ Semantic : Meaning of word
  - ↳ 3 + "hi" => Static semantic error

Scanned with CamScanner